

Informe de Desarrollo de Software – abril 2026

Atuntaqui, 13 de abril de 2026

Desarrollo de sistema – Auditoría Enterprise-Grade (SIGEPAA)

El siguiente documento detalla las actividades técnicas realizadas para la implementación del sistema de trazabilidad empresarial y los estándares de seguridad operativa en el proyecto SIGEPAA, correspondientes al periodo de Marzo de 2026. Basado en los paradigmas de Arquitectura Limpia, Compliance Estratégico y Monitoreo Proactivo, este informe certifica la robustez, testeabilidad y escalabilidad de la base de datos (PostgreSQL 18), garantizando un resguardo de información sostenible y blindado ante alteraciones no autorizadas.

1. Visión Arquitectónica: Auditoría en SIGEPAA

El proyecto SIGEPAA es una plataforma crítica para la gestión integral, requiriendo mecanismos auditables que optimicen tanto el compliance tributario/gubernamental como la rendición de cuentas. El pilar fundamental de su diseño de auditoría es la inmutabilidad y eficiencia: el sistema se desacopla a nivel de motor de datos (schema *audit*) a través de triggers automáticos e inyección de contexto. La dirección de los registros fluye unidireccionalmente, salvaguardando la información histórica frente a cualquier brecha en el backend.

La arquitectura de auditoría es una solución robusta que segmenta responsabilidades de persistencia, alertas e histórico. Esta separación facilita la evolución estructural, garantizando escalabilidad a largo plazo mediante particionamiento dinámico. En SIGEPAA, este estándar se aplica transversalmente en las operaciones de 25 tablas críticas, asegurando la integridad transaccional y excelencia técnica.

1.1 Estructura de Componentes Funcionales

La solución se organiza en engranajes operativos dentro de la base de datos:

- **Configuración y Metadatos (*tabla_config*):** Define qué nivel de trazabilidad aplica a cada modelo.
- **Motor de Intercepción (Triggers):** Captura automática (AFTER INSERT/UPDATE/DELETE) y enmascaramiento criptográfico de cambios.
- **Repositorio Particionado (*registro*):** Almacenamiento maestro en tablas mensuales independientes optimizadas.
- **Capa de Vigilancia (*alertas*):** Monitoreo cronológico y tipificación automática de anomalías.
- **Capa Analítica (*Vistas*):** Extracción limpia de diferencias JSON (*vw_cambios_recientes*, *vw_historial_fila*, etc.).

Cada uno de estos componentes de base de datos y backend operan en sinergia bajo niveles precisos de abstracción:

1. **Capa de Motor (PostgreSQL) - El Interceptor:** La capa más reactiva. Administra triggers y funciones *fn_registrar* que capturan las versiones estructurales inmutables *OLD* y *NEW*.
2. **Capa de Enmascaramiento y Seguridad - El Blindaje:** Responsable de ofuscar vía SHA-256 columnas sensibles (tokens y credenciales) directamente en las inserciones para cumplimiento normativo.
3. **Capa de Middleware (Backend NestJS) - El Contexto:** Provee los datos enriquecidos provenientes de los clientes HTTP (Inyección del *usuario_id*, *X-Forwarded-For* JWT *jti*).
4. **Capa de Cumplimiento (Retención) - El Archivista:** Funciones como *fn_limpiar_particiones* actúan cíclicamente rigiendo retenciones normadas (ej: 7 años según directrices SRI).

2. Guía de Integración y Operación

Para mantener la coherencia y el registro estandarizado en la empresa, la anexión de nuevos módulos al sistema de auditoría debe ejecutar el siguiente flujo de trabajo:

Fase A: Configuración del Dominio Auditable

1. **Registro de Nivel:** Inclusión en *audit.tabla_config* de las entidades estableciendo la profundidad de escáner (*MINIMAL*, *STANDARD* o *FULL*).
2. **Exclusión Controlada:** Exclusión de columnas técnicas internas o reajuste de datos enmascarables.
La pureza de este paso es vital: determina el balance entre completitud del historial financiero vs carga transaccional del disco.

Fase B: Implementación de Escuchadores (Infraestructura)

1. **Despliegue Dinámico:** Ejecución de la rutina idempotente *fn_aplicar_triggers()* construyendo los eventos reactivos sobre las tablas de producción (sin latencia perjudicial ni bloqueos agresivos).
Esta capa abstrae el proceso de conexión; permitiendo pausar funcionalmente la telemetría en migraciones masivas para salvaguardar CPU con *fn_pausar_tabla*.

Fase C: Lógica de Inyección de Sesión (Aplicación / NestJS)

1. **Context Injector:** Implementación de clases *AuditInterceptor* ó *AuditMiddleware* a nivel del Request.
2. **Transaccionalidad:** Ejecución forzada de *SELECT audit.fn_set_contexto()* previo al primer mutador transaccional.

Fase D: Explotación y Monitoreo (Presentación / Analítica)

1. **Paneles BI:** Consultas optimizadas directas a las vistas *vw_estadisticas_generales* y visores en el Frontend.
2. **Resolución de Alertas:** Control de ciclo de vida en incidentes con *fn_resolver_alerta()*.

Nota: Procesos ejecutados en la migración de la base de datos. en el Frontend se encuentra en proceso de desarrollo e implementacion de los paneles BI y la resolucion de alertas.

Proceso de elaboración de la Auditoría

1. Creación del schema *audit*

El schema *audit* es el contenedor de todas las funciones, tablas y vistas relacionadas con la auditoría. Es un schema independiente que se crea para separar la lógica de auditoría de la lógica de negocio.

```
CREATE SCHEMA IF NOT EXISTS audit;
```

```
COMMENT ON SCHEMA audit IS
```

```
'Schema de auditoría enterprise – grade para SIGEPAA.'
```

```
'Registra INSERT/UPDATE/DELETE de todas las tablas críticas con diff JSONB,'
```

```
'contexto de sesión, alertas automáticas y retención por particionado mensual.'
```

```
-- Tipo enumerado para nivel de auditoría
```

```
DO $$
```

```
BEGIN
```

```
IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'audit_nivel' AND typnamespace  
= (SELECT oid FROM pg_namespace WHERE nspname = 'audit')) THEN
```

```
CREATE TYPE audit.audit_nivel AS ENUM ('MINIMAL', 'STANDARD', 'FULL');
```

```
END IF;
```

```
END$$;
```

```
DO $$
```

```
BEGIN
```

```
IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname  
= 'audit_operacion' AND typnamespace  
= (SELECT oid FROM pg_namespace WHERE nspname = 'audit')) THEN
```

```
CREATE TYPE audit.audit_operacion AS ENUM ('INSERT', 'UPDATE', 'DELETE', 'TRUNCATE');
```

```
END IF;
```

```
END$$;
```

```
DO $$
```

```
BEGIN
```

```
IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname  
= 'audit_severidad' AND typnamespace  
= (SELECT oid FROM pg_namespace WHERE nspname = 'audit')) THEN
```

```
CREATE TYPE audit.audit_severidad AS ENUM ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL');
```

```
END IF;
```

```
END$$;
```

2. Creación de tablas de registro

La tabla *audit.registro* es la tabla central de auditoría. Es una tabla particionada por mes que almacena todos los eventos de auditoría.

```
CREATE TABLE IF NOT EXISTS audit.registro (
```

```
-- Identificación del evento
```

```
audit_id BIGINT GENERATED ALWAYS AS IDENTITY,
```

```
audit_timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
-- ¿Quién realizó la acción?
```

```

usuario_id      UUID,          -- app.usuario_id (SET LOCAL)
usuario_nombre  TEXT,          -- app.usuario_nom
ip_address      INET,          -- app.ip_address
app_nombre      TEXT          DEFAULT 'SIGEPAA',
sesion_id       TEXT,          -- app.sesion_id (JWT jti, etc.)

-- ¿Qué tabla/registro fue afectado?
schema_nombre   TEXT          NOT NULL DEFAULT 'public',
tabla_nombre    TEXT          NOT NULL,
operacion       audit.audit_operacion NOT NULL,
pk_valor        JSONB         NOT NULL,    -- {col_pk: valor,...}

-- Datos antes/después (según nivel configurado)
datos_antes     JSONB,         -- NULL en INSERT
datos_despues   JSONB,         -- NULL en DELETE
campos_cambiados TEXT[]       DEFAULT '{}',
diff_jsonb      JSONB         DEFAULT '{}':JSONB, -- {campo: {antes: x, despues: y}}

-- Trazabilidad técnica
query_hash      TEXT,          -- md5 de current_query() para análisis
duracion_ms     NUMERIC(12,4),
metadata        JSONB         DEFAULT '{}':JSONB, -- datos adicionales libres

PRIMARY KEY (audit_id, audit_timestamp)
) PARTITION BY RANGE (audit_timestamp);

COMMENT ON TABLE audit.registro IS
'Registro central de auditoría enterprise. Particionada mensualmente.'
'Append – only: UPDATE y DELETE están protegidos por RLS y permisos de rol.';

COMMENT ON COLUMN audit.registro.pk_valor IS
'Valor de la clave primaria de la fila afectada, siempre en formato JSONB '
'para soportar PKs compuestas. Ej: {"factura_id": 42}';
COMMENT ON COLUMN audit.registro.diff_jsonb IS
'Diferencial columna a columna en UPDATE. Formato: '
'{"columna": {"antes": valor_old, "despues": valor_new}}.'
'Vacío en INSERT y DELETE.';
COMMENT ON COLUMN audit.registro.campos_cambiados IS
'Array de nombres de columnas que realmente cambiaron en un UPDATE.'
'Permite búsqueda rápida sin parsear diff_jsonb.';

-- Índices en la tabla madre
-- Búsqueda por tabla + tiempo (el caso más frecuente)
CREATE INDEX IF NOT EXISTS idx_audit_reg_tabla_ts
ON audit.registro (tabla_nombre, audit_timestamp DESC);

-- Búsqueda por usuario + tiempo
CREATE INDEX IF NOT EXISTS idx_audit_reg_usuario_ts
ON audit.registro (usuario_id, audit_timestamp DESC);

-- Búsqueda por tipo de operación

```

```

CREATE INDEX IF NOT EXISTS idx_audit_reg_operacion
  ON audit.registro (operacion, audit_timestamp DESC);

-- Búsqueda dentro del PK (ej: todos los eventos de factura_id = 99)
CREATE INDEX IF NOT EXISTS idx_audit_reg_pk_gin
  ON audit.registro USING GIN (pk_valor);

-- Búsqueda dentro del diff (ej: cambios en columna "estado")
CREATE INDEX IF NOT EXISTS idx_audit_reg_diff_gin
  ON audit.registro USING GIN (diff_jsonb);

-- Búsqueda inversa cronológica global
CREATE INDEX IF NOT EXISTS idx_audit_reg_ts_desc
  ON audit.registro (audit_timestamp DESC);

-- Búsqueda por sesión (correlación con logs de app)
CREATE INDEX IF NOT EXISTS idx_audit_reg_sesion
  ON audit.registro (sesion_id) WHERE sesion_id IS NOT NULL;

```

3. Creación de Particiones

Las particiones son tablas físicas que se crean para almacenar los datos de la tabla *audit.registro*. Cada partición almacena los datos de un mes específico.

```

CREATE TABLE IF NOT EXISTS audit.registro_2024_01 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 01 - 01') TO ('2024 - 02 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_02 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 02 - 01') TO ('2024 - 03 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_03 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 03 - 01') TO ('2024 - 04 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_04 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 04 - 01') TO ('2024 - 05 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_05 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 05 - 01') TO ('2024 - 06 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_06 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 06 - 01') TO ('2024 - 07 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_07 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 07 - 01') TO ('2024 - 08 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_08 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 08 - 01') TO ('2024 - 09 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_09 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 09 - 01') TO ('2024 - 10 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_10 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 10 - 01') TO ('2024 - 11 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_11 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 11 - 01') TO ('2024 - 12 - 01');
CREATE TABLE IF NOT EXISTS audit.registro_2024_12 PARTITION OF audit.registro FOR
VALUES FROM ('2024 - 12 - 01') TO ('2025 - 01 - 01');
-- ...

```

-- DEFAULT PARTITION -- captura fechas fuera de rango

```
CREATE TABLE IF NOT EXISTS audit.registro_default PARTITION OF audit.registro DEFAULT;
COMMENT ON TABLE audit.registro_default IS
```

```
'Partición de seguridad. Si hay datos aquí, significa que falta crear '
```

```
'la partición mensual correspondiente. Ejecutar audit.fn_crear_particion_mes().';
```

```
-----
-- Función auxiliar: crear partición para un mes específico dinámicamente
-----
```

```
CREATE OR REPLACE FUNCTION audit.fn_crear_particion_mes(p_fecha DATE DEFAULT
CURRENT_DATE)
```

```
RETURNS TEXT
```

```
LANGUAGE plpgsql
```

```
SECURITY DEFINER
```

```
SET search_path = audit, public
```

```
AS $$
```

```
DECLARE
```

```
    v_inicio DATE;
```

```
    v_fin DATE;
```

```
    v_nombre TEXT;
```

```
    v_sql TEXT;
```

```
BEGIN
```

```
    v_inicio := DATE_TRUNC('month', p_fecha)::DATE;
```

```
    v_fin := (v_inicio + INTERVAL '1 month')::DATE;
```

```
    v_nombre := 'audit.registro_' || TO_CHAR(v_inicio, 'YYYY-MM');
```

```
-- Verificar si ya existe
```

```
IF EXISTS (
```

```
    SELECT 1 FROM pg_tables
```

```
    WHERE schemaname = 'audit'
```

```
    AND tablename = 'registro_' || TO_CHAR(v_inicio, 'YYYY-MM')
```

```
) THEN
```

```
    RETURN 'YA EXISTE: ' || v_nombre;
```

```
END IF;
```

```
v_sql := FORMAT(
```

```
    'CREATE TABLE %s PARTITION OF audit.registro '
```

```
    'FOR VALUES FROM (%L) TO (%L)',
```

```
    v_nombre, v_inicio, v_fin
```

```
);
```

```
EXECUTE v_sql;
```

```
RETURN 'CREADA: ' || v_nombre || ' (' || v_inicio || ' → ' || v_fin || ')';
```

```
EXCEPTION WHEN OTHERS THEN
```

```
    RETURN 'ERROR: ' || SQLERRM;
```

```
END;
```

```
$$;
```

```
COMMENT ON FUNCTION audit.fn_crear_particion_mes(DATE) IS
```

```
'Crea la partición mensual de audit.registro para la fecha dada.'
```

```
'Llamar mensualmente (ej: primer día del mes anterior al siguiente).'
```

```
'Ejemplo: SELECT audit.fn_crear_particion_mes('2028-01-01');
```

```
-- Función: crear las próximas N particiones desde hoy
CREATE OR REPLACE FUNCTION audit.fn_crear_proximas_particiones(p_meses INTEGER
DEFAULT 3)
RETURNS SETOF TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit,public
AS $$
DECLARE
    i INTEGER;
BEGIN
    FOR i IN 0..p_meses LOOP
        RETURN NEXT audit.fn_crear_particion_mes(
            (DATE_TRUNC('month',CURRENT_DATE) + (i || ' months')::INTERVAL)::DATE
        );
    END LOOP;
END;
$$;

COMMENT ON FUNCTION audit.fn_crear_proximas_particiones(INTEGER) IS
    'Crea las próximas N particiones mensuales desde el mes actual.'
    'Recomendado: ejecutar mensualmente con pg_cron o cron del SO.'
    'Ejemplo: SELECT audit.fn_crear_proximas_particiones(3);'
```

4. Configuración por Tabla (*tabla_config*)

Esta tabla funciona como el panel de control de la auditoría. Nos permite decidir de forma directa qué información guardar de cada tabla, qué nivel de detalle queremos (*MINIMAL*, *STANDARD* o *FULL*), y qué datos sensibles (como contraseñas) deben ocultarse.

```
CREATE TABLE IF NOT EXISTS audit.tabla_config (
    tabla_nombre TEXT PRIMARY KEY,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    nivel audit.audit_nivel NOT NULL DEFAULT 'STANDARD',
    columnas_excluidas TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[],
    columnas_enmascarar TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[],
    auditar_insert BOOLEAN NOT NULL DEFAULT TRUE,
    auditar_update BOOLEAN NOT NULL DEFAULT TRUE,
    auditar_delete BOOLEAN NOT NULL DEFAULT TRUE,
    retener_dias INTEGER CHECK (retener_dias > 0)
    -- ...
);
```

Beneficio clave: Si necesitamos hacer una gran importación de datos y no queremos generar miles de registros de auditoría innecesarios, podemos pausar la auditoría temporalmente aquí mismo. Además, si mañana decidimos que una nueva columna es confidencial, la ocultamos de inmediato sin tener que modificar el código de la aplicación.

5. Registro de la Sesión del Usuario

Para que la auditoría sea verdaderamente útil, necesitamos saber *quién* hizo el cambio. Como la base de datos por sí sola no sabe qué usuario de la página web está conectado, creamos dos funciones de apoyo:

1. *audit.fn_set_contexto()*: Permite que nuestra aplicación le envíe a la base de datos la información vital del usuario justo antes de hacer cualquier modificación: su identificador, su IP y desde qué sesión se conecta.
2. *audit.fn_enmascarar_jsonb()*: Se encarga de ocultar las contraseñas u otros datos confidenciales transformándolos en códigos ilegibles.

6. Función Principal de Registro (*fn_registrar*)

Es el motor invisible de la auditoría. Cada vez que se crea, edita o elimina algún dato, esta función entra en acción para:

- Revisar cómo estaban los datos "antes" y cómo quedaron "después".
- Identificar exactamente qué cambió (para solo guardar esa pequeña diferencia).
- Ocultar la información sensible.
- Guardar el reporte ordenadamente en la tabla correspondiente al mes actual.

Todo esto ocurre en segundo plano de manera segura. Si por algún motivo la auditoría falla, el sistema prioriza que la empresa siga operando con normalidad.

7. Activador Automático de Auditoría (*fn_aplicar_triggers*)

Para no tener que programar manualmente tabla por tabla cada vez que crecemos, creamos un instalador automático. Cuando queremos auditar una herramienta nueva en el sistema, solo escribimos el nombre de su base en la configuración y ejecutamos:

```
SELECT * FROM audit.fn_aplicar_triggers();
```

La base de datos se encargará de configurar y preparar todo instantáneamente para empezar a auditar esa herramienta.

8. Sistema Inteligente de Alertas

Tener un registro de cientos de cambios diarios sirve poco si nadie avisa de eventos peligrosos. Por eso, el sistema cuenta con "vigilantes" o alertas automáticas.

El sistema evalúa constantemente lo que hacen los usuarios. Algunos ejemplos de lo que atrapa nuestro panel:

- **Borrado Masivo:** Da aviso si alguien elimina más de 10 registros en menos de 5 minutos.
- **Cambio crítico:** Notifica de inmediato si alguien altera una tarifa o sus reglas.
- **Escalada de Autoridad:** Alerta si a un usuario se le otorgan permisos de administrador sorpresivamente.

Cada alerta tiene una prioridad (Baja, Media, Alta, Crítica) y exigen ser analizadas y justificadas.

9. Vistas Resumidas (Reportes Gerenciales y de Análisis)

Los datos guardados se organizan en "vistas preconfiguradas" fáciles de leer para no tener que crear complejos reportes desde cero.

- *vw_actividad_usuario*: Permite ver rápidamente cuántas acciones ha realizado un usuario en un día.
- *vw_cambios_recientes*: Alimenta las pantallas para ver "en vivo" cada modificación o trabajo que se está realizando en la empresa.
- *vw_alertas_activas*: Muestra los incidentes de seguridad que faltan por revisar.
- *vw_historial_fila*: Sumamente útil para revisar el historial aislado de un dato o papel fundamental. Ejemplo: permite ver en orden todas las fechas en que la *Factura N° 42* fue modificada y quiénes lo hicieron consecutivamente.

10. Seguridad y Accesos por Rol

Se estableció un enfoque estricto: ni los encargados de auditar o de revisar, ni los mismos programadores pueden alterar o borrar el historial. Todo proceso está blindado.

Existen perfiles de permisos estandarizados en la base de datos:

- **Lector**: Solo pueden ver y generar los historiales que les competen. Ideal para personal externo, jefes de área, evaluadores de métricas.
- **Automático**: Un perfil de sistema oculto capaz únicamente de sumar datos a la bitácora al transitar cambios correctos.
- **Administrador**: Autorizado únicamente para modificar o adaptar las reglas de seguridad de este archivo general.

Incluso si un usuario entra a propósito para limpiar evidencias de su corrupción perdiendo una base, topará de frente con una programación transversal configurada que bloquea permanentemente a cualquier usuario el comando de borrar o truncar la bitácora histórica.

11. Limpieza Automática (Cumplimiento de Ley y Almacenamiento)

Por regla u orden tributaria (Ej. SRI / Comprobantes Electrónicos), la evidencia vital de información se debe guardar generalmente por un periodo cercano de 6 a 7 años.

El sistema incluye una función natural de "auto-limpieza", esta se ejecuta discretamente cada final de mes y revisa si existen carpetas o particiones pasadas de este sistema que han excedido su uso legal obligatorio y las expulsa definitivamente del servidor de forma limpia.

Dicha directriz garantiza el cumplimiento legal íntegro a favor del Estado, pero previene tajantemente que el historial sature los discos duros, manteniendo alta la frescura, el espacio y la velocidad operativa para todo el negocio del SIGEPAA.

12. Captura de la Sesión en el Servidor (Interceptores del Backend)

Para que la base de datos sepa exactamente quién, desde dónde y cómo se realizan los cambios de forma automática, las capas de la aplicación se aseguran de recolectar esta información sin

interrumpir a los usuarios. Esto se logra mediante dos "aduanas" de información (Interceptores) que controlan la conexión:

12.1 La Entrada y Salida Principal (Client Gateway)

Cada vez que un usuario, empleado o intruso se comunica con nuestros sistemas, el interceptor inteligente (*AuditContextInterceptor*) funciona como la primera línea general de vigilancia.

Qué revisa obligatoriamente:

Siempre empaqueta la Identidad del trabajador (si es que existe), la dirección o ubicación de su equipo (IP), el registro técnico del dispositivo y su sesión temporal. Este proceso de captura ocurre **incluso antes** de que el usuario logre iniciar sesión con éxito. Gracias a esta regla, si un atacante intenta adivinar una contraseña, el sistema guardará un rastro exacto desde dónde vino ese ataque fallido identificando a su origen sin que pase desapercibido.

A nivel de código, se implementó el método intercept() en la clase AuditContextInterceptor. Su función clave es extraer de forma obligatoria la IP y el registro del dispositivo navegador (incluso si el usuario es sospechoso y aún no tiene permisos) y empaquetar todo temporalmente en la memoria activa del sistema.

```
intercept(context: ExecutionContext, next: CallHandler): Observable < any > {
  const request = context.switchToHttp().getRequest();
  const user = request['user'];

  const ip = request.headers['x - forwarded
    - for']?.toString() || request.socket.remoteAddress || '0.0.0.0';
  const userAgent = request.headers['user - agent']?.toString() || 'Unknown';

  return new Observable((observer) => {
    AuditContextStorage.run(
      {
        userId: user?.sub,
        userName: user?.username || 'Anonymous',
        ip,
        sessionId: user?.jti || request.cookies?.['auth_token']?.slice(0, 10),
        userAgent,
      },
      () => {
        next.handle().subscribe(observer);
      },
    );
  });
}
```

12.2 Servicio de Gestión de Identidad y Acceso (Authentication Microservice)

Debido a que el SIGEPAA tiene una arquitectura distribuida (escalable), casi siempre está distribuida en varios "microsistemas" pequeños para no saturarse. El peligro de estos tipos de plataforma es perder el hilo o rastro de la persona original durante el brinco de la petición inicial hacia otro módulo alojado más a fondo.

Para evitar trabajar con usuarios "fantasmas", introducimos al interceptor de respaldo (*AuditInterceptor*). Es el responsable de que cualquier instrucción o solicitud interna (por ejemplo, mediante mensajería Kafka) viaje y obligue a cargar consigo la "etiqueta de identificación" original. Sin importar si un proceso pasa por 5 componentes ocultos, ni el sistema ni la base de datos olvidarán jamás quién ordenó esa acción inicialmente, blindando la auditoría corporativa.

Para respaldar la trazabilidad secreta, se implementó el método `intercept()` dentro de la clase `AuditInterceptor`. Este método reacciona al momento de brincar entre microservicios: desempaca los datos de la "carga técnica" original (como el identificador de la persona y su IP real) y reactiva el contexto. De esta manera asegura que el microservicio aislado recuerde siempre visualmente quién inició la orden desde el principio, imposibilitando los borrados anónimos.

```
intercept(context: ExecutionContext, next: CallHandler): Observable < any > {
  if (context.getType() === 'rpc') {
    const rpcContext = context.switchToRpc();
    const kafkaContext = rpcContext.getContext < KafkaContext > ();
    const headers = kafkaContext.getMessage().headers;

    const userId = headers?.['user - id']?.toString();
    const userName = headers?.['user - name']?.toString();
    const ip = headers?.['user - ip']?.toString();
    const sessionId = headers?.['user - session - id']?.toString();
    const userAgent = headers?.['user - agent']?.toString();

    if (userId || ip || sessionId) {
      return new Observable((observer) => {
        AuditContextStorage.run(
          {
            userId: userId || undefined,
            userName: userName || 'Anonymous',
            ip: ip || '0.0.0.0',
            sessionId: sessionId || undefined,
            userAgent: userAgent || 'N/A',
          },
          () => { next.handle().subscribe(observer); }
        );
      });
    }
  }
  return next.handle();
}
```

13. Exposición de la Auditoría a los Usuarios (Client Gateway Controller)

Almacenar los datos de forma segura e impecable es vital, pero también debemos permitir que los administradores, gerentes o auditores externos puedan revisar esa información cómoda y visualmente desde la plataforma web sin tener que entrar directamente a revisar el código fuente o manipular los servidores.

Para lograr esto, creamos dentro del **Client Gateway** un "Controlador" para el módulo de auditoría.

Su función es ser un puente oficial y sumamente seguro entre las ventanas informativas (la pantalla del navegador del usuario) y los registros protegidos del sistema.

Qué hace exactamente:

1. **Registro de Asistencias:** Provee un salvoconducto seguro (*/log-session*) para asentar cada vez que alguien ingresa, sale o fracasa vergonzosamente al intentar iniciar sesión en el SIGEPAA.
2. **Extractor de Movimientos:** Permite pedir reportes visuales (*/get-logs*) a la enorme base de datos para entregar a pantalla exactamente lo que el jefe solicita. Por ejemplo: "*¿Qué cambios ocurrieron ayer en los Clientes?*"
3. **Control de Entrada del Personal:** Habilita el extraer con sumo perfeccionamiento (*/get-session-logs*) los apuntes detallados sobre cuándo los empleados operaron, iniciaron o cerraron sistemas.

Todo este flujo y control de tráfico hacia el cajón confidencial está fuertemente resguardado por Validadores de Seguridad (AuthGuards) y viaja en rutas subterráneas privadas del sistema (Kafka). Literalmente ninguna persona que no posea un Carné de Identidad digital activo y avalado del SIGEPAA puede tan si quiera tocar este módulo.

Para soportar dinámicamente toda esta infraestructura visual hacia adelante, se implementó el Controlador especializado *AuditGatewayController*, enjaulado obligatoriamente por el estricto guardián de permisos del sistema (*AuthGuard*).

Dentro de esta estructura principal se construyeron tres métodos inquebrantables que sirven como túneles de información (gracias a Kafka *clientKafka.send*):

1. **Método *logSession()*:** Habilita la ruta HTTP protegida *POST /log-session* encargada de apuntar en los libros de vigilancia cada vez que ocurren cierres, aperturas exitosas o fallos críticos de sesión por parte de los trabajadores.

```

@Post('log - session')
@ApiOperation({
  summary: 'Registrar un evento de sesión (Login/Logout)',
  description: 'Guarda un registro detallado de intentos de inicio o cierre de sesión.'
})
async logSession(
  @Body() requestData: LogSessionRequest,
  @Req() request: Request,
): Promise < ApiResponse > {
  try {
    const response = await sendKafkaRequest(
      this.clientKafka.send('audit.log - session', requestData),
    );

    return new ApiResponse(
      'Session event logged successfully',
      response,
      request.url,
    );
  } catch (error) {
    throw new RpcException(error);
  }
}
  
```

}

2. **Método `getAuditLogs()`:** Configura el puerto de lectura `GET /get-logs`. Procesa y valida únicamente las peticiones estructuradas de los directivos, entregándoles informes detallados acerca de qué campo exacto de una tabla fue vulnerado o cambiado, paginándolo limpiamente.

```
@Get('get - logs')
@ApiOperation({
  summary: 'Obtener historial de mutaciones de base de datos',
  description: 'Retorna los registros de auditoría de tablas (INSERT/UPDATE/DELETE).',
})
async getAuditLogs(
  @Query('limit') limit: number,
  @Query('offset') offset: number,
  @Query('tableName') tableName: string,
  @Query('operation') operation: 'INSERT' | 'UPDATE' | 'DELETE' | 'TRUNCATE',
  @Query('userId') userId: string,
  @Req() request: Request,
): Promise < ApiResponse > {
  try {
    const payload: GetAuditLogsRequest = {
      limit: limit ? Number(limit) : undefined,
      offset: offset ? Number(offset) : undefined,
      tableName,
      operation,
      userId,
    };

    const response = await sendKafkaRequest(
      this.clientKafka.send('audit.get - logs', payload),
    );

    return new ApiResponse(
      'Audit logs retrieved successfully',
      response,
      request.url,
    );
  } catch (error) {
    throw new RpcException(error);
  }
}
```

3. **Método `getSessionLogs()`:** A través de la vía `GET /get-session-logs`, los encargados de Recursos Humanos o Administradores pueden extraer reportes sobre asistencias o ingresos, filtrando variables puntuales (Ej: Listarme solo intentos de `LOGIN_FAILED` atados a un usuario específico durante el día de hoy).

```
@Get('get - session - logs')
@ApiOperation({
```

```
summary: 'Obtener historial de sesiones',
description: 'Retorna los registros de los eventos de inicio o falla de sesión de usuarios.',
})
async getSessionLogs(
  @Query('limit') limit: number,
  @Query('offset') offset: number,
  @Query('userId') userId: string,
  @Query('event') event: 'LOGIN' | 'LOGOUT' | 'LOGIN_FAILED',
  @Req() request: Request,
): Promise < ApiResponse > {
  try {
    const payload: GetSessionLogsRequest = {
      limit: limit ? Number(limit) : undefined,
      offset: offset ? Number(offset) : undefined,
      event,
      userId,
    };

    const response = await sendKafkaRequest(
      this.clientKafka.send('audit.get-session-logs', payload),
    );

    return new ApiResponse(
      'Session logs retrieved successfully',
      response,
      request.url,
    );
  } catch (error) {
    throw new RpcException(error);
  }
}
```

14. Interfaz Visual (Módulo de Auditoría en el Frontend)

Para brindar al equipo gerencial y de cumplimiento (compliance) una herramienta cómoda de análisis sin acudir a consolas oscuras ni consultas SQL, se construyó un módulo netamente visual en el Frontend de la plataforma (basado en Arquitectura Limpia).

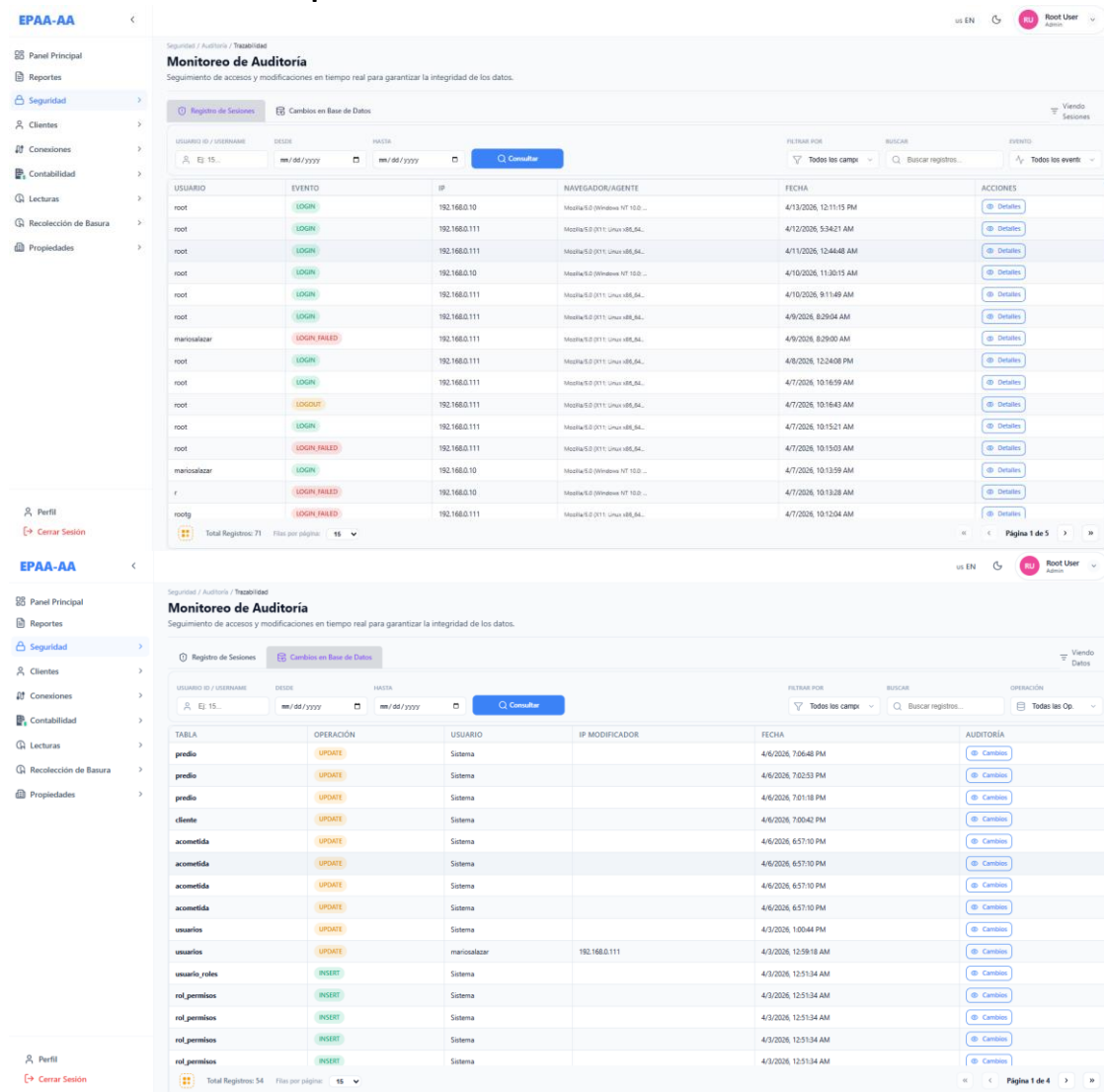
Este entorno consta de diferentes paneles dinámicos (*AuditDashboard*) que consumen de manera segura las rutas habilitadas por nuestro servidor, mostrando al usuario directivo una interfaz ordenada, filtrable y paginada en tiempo real.

Componentes Principales de la Vista:

1. **Histórico de Cambios en Sistema (*DataLogsTable*):** Componente que lista cada edición reportada por la base de datos (PostgreSQL), desenmarañando visualmente la variable "Antes" y "Después" para que un humano identifique rápidamente qué información alteró, borró o modificó puntualmente un empleado.
2. **Panel de Rastros de Acceso (*SessionLogsTable*):** Permite de un vistazo corroborar fechas precisas, horarios y credenciales de las personas que ingresaron al sistema desde sus dispositivos, atrapando incluso **intentos de conexión fallidos por contraseña**.

Toda esta visualización se conecta a través de un Contexto de Aplicación (*AuditContext*) que encapsula ordenadamente toda la comunicación (REST) extrayendo la bitácora sin entorpecer ni retrasar visualmente al resto de módulos con los que convive diariamente el empleado.

Evidencia de Interfaz Operando:



Monitoreo de Auditoría
Seguimiento de accesos y modificaciones en tiempo real para garantizar la integridad de los datos.

Registro de Sesiones

USUARIO ID / USERNAME	FECHA	IP	NAVEGADOR/AGENTE	FECHA	ACCIONES
root	LOGIN	192.168.0.10	Mozilla/5.0 (Windows NT 10.0...	4/13/2026, 12:11:15 PM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/12/2026, 5:34:21 AM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/11/2026, 12:44:48 AM	Detalles
root	LOGIN	192.168.0.10	Mozilla/5.0 (Windows NT 10.0...	4/10/2026, 11:30:15 AM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/10/2026, 9:11:49 AM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/9/2026, 8:29:04 AM	Detalles
mariscalcacer	LOGIN FAILED	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/9/2026, 8:29:00 AM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/8/2026, 12:34:08 PM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/7/2026, 10:16:59 AM	Detalles
root	LOGOUT	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/7/2026, 10:16:43 AM	Detalles
root	LOGIN	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/7/2026, 10:15:21 AM	Detalles
root	LOGIN FAILED	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/7/2026, 10:15:03 AM	Detalles
mariscalcacer	LOGIN	192.168.0.10	Mozilla/5.0 (Windows NT 10.0...	4/7/2026, 10:13:59 AM	Detalles
r	LOGIN FAILED	192.168.0.10	Mozilla/5.0 (Windows NT 10.0...	4/7/2026, 10:13:28 AM	Detalles
root	LOGIN FAILED	192.168.0.111	Mozilla/5.0 (Linux x86_64...	4/7/2026, 10:12:04 AM	Detalles

Monitoreo de Auditoría
Seguimiento de accesos y modificaciones en tiempo real para garantizar la integridad de los datos.

Registro de Sesiones

TABLA	OPERACIÓN	USUARIO	IP MODIFICADOR	FECHA	AUDITORIA
predio	UPDATE	Sistema		4/6/2026, 7:06:48 PM	Cambios
predio	UPDATE	Sistema		4/6/2026, 7:02:53 PM	Cambios
predio	UPDATE	Sistema		4/6/2026, 7:01:18 PM	Cambios
cliente	UPDATE	Sistema		4/6/2026, 7:00:42 PM	Cambios
acometida	UPDATE	Sistema		4/6/2026, 6:57:10 PM	Cambios
acometida	UPDATE	Sistema		4/6/2026, 6:57:10 PM	Cambios
acometida	UPDATE	Sistema		4/6/2026, 6:57:10 PM	Cambios
usuarios	UPDATE	Sistema		4/3/2026, 1:00:44 PM	Cambios
usuarios	UPDATE	mariscalcacer	192.168.0.111	4/3/2026, 12:58:18 AM	Cambios
usuarios_roles	INSERT	Sistema		4/3/2026, 12:51:34 AM	Cambios
rol_permisos	INSERT	Sistema		4/3/2026, 12:51:34 AM	Cambios
rol_permisos	INSERT	Sistema		4/3/2026, 12:51:34 AM	Cambios
rol_permisos	INSERT	Sistema		4/3/2026, 12:51:34 AM	Cambios
rol_permisos	INSERT	Sistema		4/3/2026, 12:51:34 AM	Cambios

Además, se implementó un dashboard para el módulo de acometidas que permite visualizar mediante un mapa la ubicación de las acometidas y su estado con sus respectivas filtraciones. Esta parte nos permite tener un control total sobre las acometidas y su estado, además de poder ver el historial de cambios en las acometidas.

Para comenzar a utilizar el módulo de auditoría, se realizó tanto en el frontend como en el backend la implementación de las rutas necesarias para que el módulo de auditoría funcione correctamente.

- Repositorio de auditoría en el Frontend

```
import type { AuditRepository } from '../domain/repositories/AuditRepository';
import { AuditClient } from '../api/AuditClient';
import type {
  GetAuditLogsParams,
```

```

    GetSessionLogsParams,
    AuditRegistroResponse,
    AuditSessionResponse
} from '../domain/models/AuditModels';
import type { ApiResponse } from '@shared/infrastructure/api/response/ApiResponse';

export class ApiAuditRepository implements AuditRepository {
    private apiClient: AuditClient;

    constructor() {
        this.apiClient = new AuditClient();
    }

    async getAuditLogs(
        params: GetAuditLogsParams
    ): Promise < ApiResponse < AuditRegistroResponse[] >> {
        return this.apiClient.getAuditLogs(params);
    }

    async getSessionLogs(
        params: GetSessionLogsParams
    ): Promise < ApiResponse < AuditSessionResponse[] >> {
        return this.apiClient.getSessionLogs(params);
    }
}

```

- Controller de auditoría en el Backend (Microservicio)

```

@Controller('audit')
export class AuditController {
    constructor(
        private readonly logSessionUseCase: LogSessionUseCase,
        private readonly getAuditLogsUseCase: GetAuditLogsUseCase,
        private readonly getSessionLogsUseCase: GetSessionLogsUseCase
    ) {}

    @MessagePattern('audit.log - session')
    async logSession(@Payload() payload: LogSessionRequest) {
        await this.logSessionUseCase.execute(payload);
        return { success: true, message: 'Session logged.' };
    }

    @MessagePattern('audit.get - logs')
    async getAuditLogs(@Payload() payload: GetAuditLogsRequest) {
        return await this.getAuditLogsUseCase.execute(payload);
    }

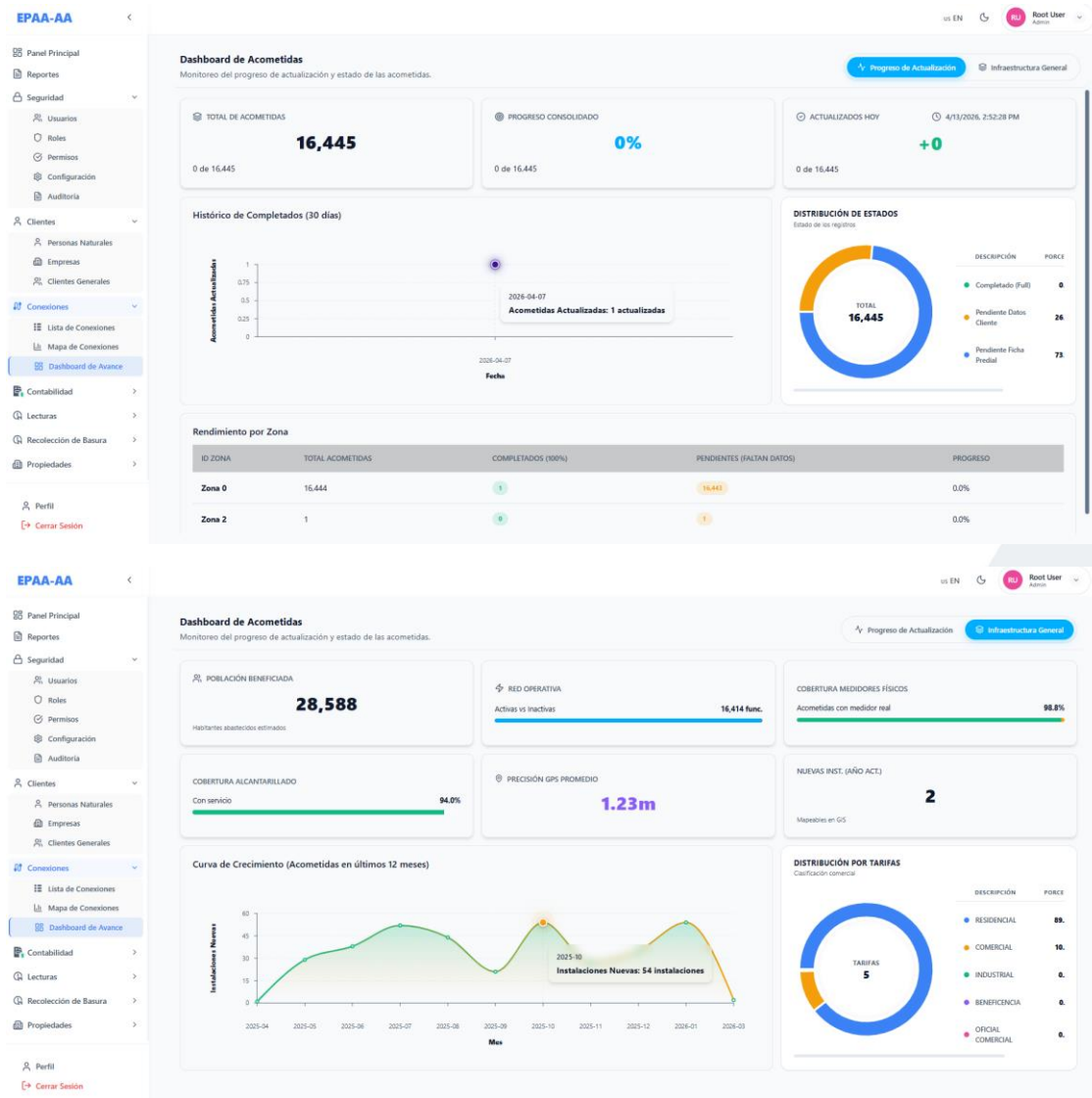
    @MessagePattern('audit.get - session - logs')
    async getSessionLogs(@Payload() payload: GetSessionLogsRequest) {

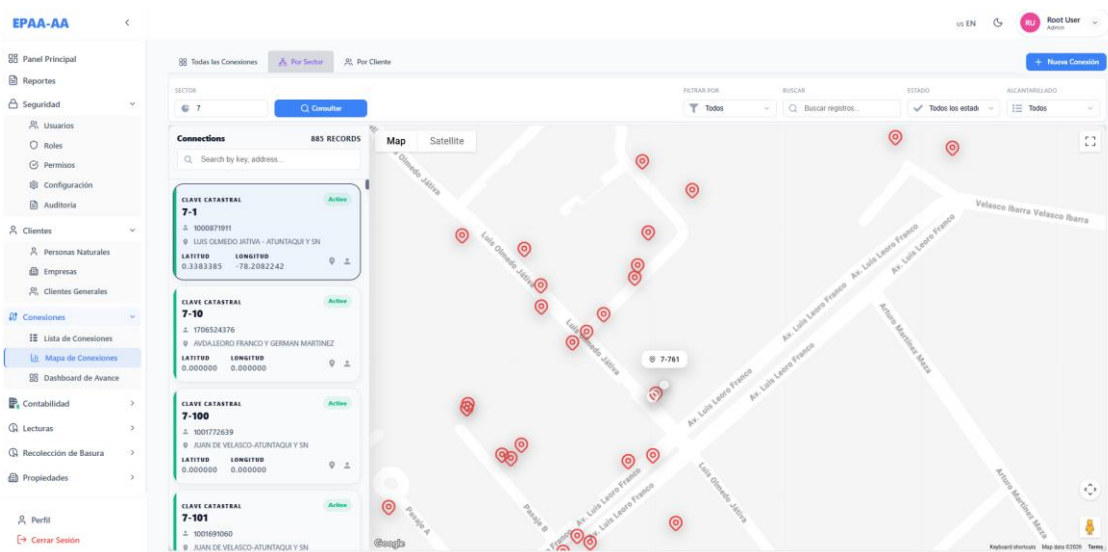
```

```

    }
    return await this.getSessionLogsUseCase.execute(payload);
  }
}

```





Mario Salazar
DESARROLLADOR DE SOFTWARE